TODOs

- [x] ~~Refine # of agents / remove "agent"s from the slides~~
- [x] ~~Plan/create full customer profile (to convey how different users use it during demo)~~
- [ ] Challenge technical questions
- [ ] CODEEEEEE
- [ ] LEARN SCRIPTS
- [x] ~~Implementation flow slide: why do we need to go through?~~

CODEEEEE

- [ ] Actions flow when we REJECT step
    - reject -> pop up a box asks why/do you want to create a new plan -> get to the reasoning/planing layer again (info about the previous risk case is saved so Manager Agent can just go back to previous layer) -> …
- [ ] In Risk Simulation, we want “The iteration history stays visible for comparison across different runs.” Currently, it is not.
- [ ] Risk Simulation and chatbot are generating very slowly. Try?
- [ ] Validate supply chain knowledge of the reasoning layer
- [ ] Extend the Actions tab so the agent doesn't stop at preparing documents but also plans delivery routes via interactive maps?
- [ ] Refactor our agents (to the new architecture)

TECHNICAL QUESTIONS

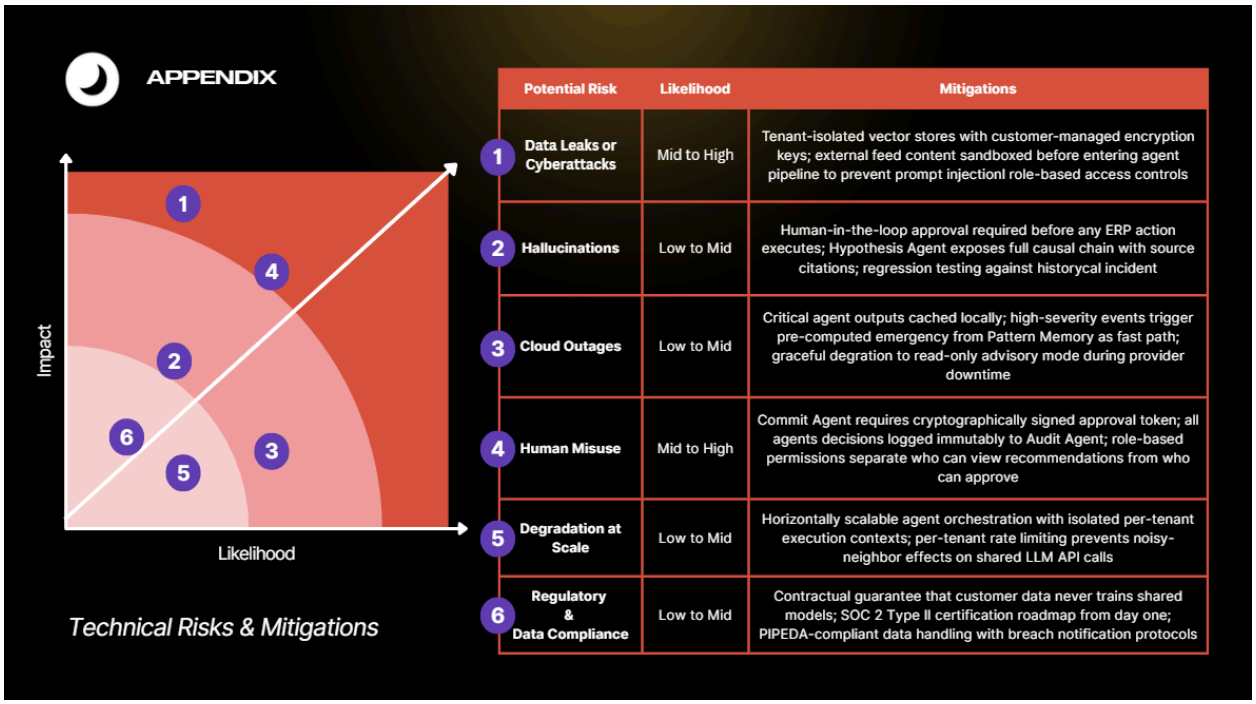
Q	A
Why this current stack? Pros & cons? Efficiency?	
What are the considerations when scaling up the project?	
Technical risk mitigations? READ THE SLIDE (attached below) and note relevant technologies	
How to ensure the accuracy or trustworthy of the agents (in terms of generating plans)	
Database capacity	
Infrastructure for running?	
Normalizer Agent outputs a Canonical Schema. Show that schema. What fields	

does it have, what happens when a GDELT event has no geographic coordinates, and how do you handle conflicting severity ratings from two different feeds on the same event?	
Hypothesis Agent generates causal chains. What model are we using, what's our prompt structure, and how do we prevent it from generating a confident causal chain on insufficient evidence?	
Why is the Scoring Agent a separate agent and not just a deterministic function? Risk Score = Probability × Exposure × Impact is a formula. You don't need an LLM for that. Defend the decision.	
Your Memory Layer has three stores. What's the underlying technology for each? Vector DB, relational, graph? And why different stores instead of one unified store?	
What's your ground truth? How do you know when your Hypothesis Agent is wrong? If the client approves a bad recommendation and loses money, how does your system know it made a mistake and how does that feed back?	
Exposure Agent maps events to suppliers, routes, and inventory. How to control how fresh the mapping data is? What happens when a supplier changes their warehouse location and nobody tells Omni (will this happen?)?	
You require human approval before ERP commits. In a fast-moving crisis: port closure, imminent hurricane, etc. what's your expected time from signal detection to Sarah seeing a ranked plan? Give me actual numbers.	
If the client approves 95% of what the system recommends, has she stopped thinking critically or has the system earned her trust? How do you design against automation bias?	
Your entire value proposition depends on LLMs reasoning correctly about supply chain causality. But LLMs were trained on internet text, not supply chain operations. What's your	

evidence that a general-purpose LLM can reason accurately about the causal relationship between a port strike in Taiwan and your customer's inventory position in …?	

Create full customer profiles

- How do different users use it during demo?
-



	Potential Risk	Likelihood	Mitigations
1	Data Leaks or Cyberattacks	Mid to High	Tenant-isolated vector stores with customer-managed encryption keys; external feed content sandboxed before entering agent pipeline to prevent prompt injectionl role-based access controls
2	Hallucinations	Low to Mid	Human-in-the-loop approval required before any ERP action executes; Hypothesis Agent exposes full causal chain with source citations; regression testing against historycal incident
3	Cloud Outages	Low to Mid	Critical agent outputs cached locally; high-severity events trigger pre-computed emergency from Pattern Memory as fast path; graceful degration to read-only advisory mode during provider downtime
4	Human Misuse	Mid to High	Commit Agent requires cryptographically signed approval token; all agents decisions logged immutably to Audit Agent; role-based permissions separate who can view recommendations from who can approve
5	Degradation at Scale	Low to Mid	Horizontally scalable agent orchestration with isolated per-tenant execution contexts; per-tenant rate limiting prevents noisy-neighbor effects on shared LLM API calls
6	Regulatory & Data Compliance	Low to Mid	Contractual guarantee that customer data never trains shared models; SOC 2 Type II certification roadmap from day one; PIPEDA-compliant data handling with breach notification protocols

Refine # of agents / remove "agent"s from the slides

- Currently: 14 agents
- ☐ Find support data to defend our choice of having many agents
- ☐ Distinguish between **Agentic tasks** (requiring reasoning and decision-making) and **Deterministic tasks** (rule-based processing).

- **Reasoning layer:** merge Cluster Agent and Exposure Agent
  - Into 1 Risk Analyst Agent: takes raw signals → groups → maps them to business exposure
  - Need to refine the prompt + clear context

- **Action layer:**
    - Change Proposal Agent and Audit Agent don't need to be LLMs
        - Turn them into tools → simply call functions like generate erp diff / log to audit trail after it gets human approval
    - Approval Agent should be a tool, UNLESS we want feedback features to refine plans
- **Planning layer:**

cleaned:

Layer	Keep
Perception	Normalization Agent
Reasoning	
Planning	
Action	

- Implementation flow slide: why do we need to go through?
    - In general, show the judges that we didn't hardcode stuff
    - Clearer picture of the data flow in the background
    - But can probably tweak to be more storytelling by putting a persona/customer profile
    - → transition from the demo (how the customer interacts on the surface) to the architecture plans (how the background layers work)
    - The time for this part could be shortened if amy spoke faster (will)
    - Remove repeating the tech stack (python, fast api, supabase) -> more business friendly + won't get technical judges bored.
    - TODO: prepare tech question: how to expand this tech stack as the project scales up? Costs? Security?